

否存在没有指明来源或去向的数据流，是否存在没有指明数据存储或所属数据流的数据元素，加工逻辑与输入的数据元素是否匹配，是否存在没有输入或输出的数据存储等。

3. 数据字典的管理

为了保证数据的一致性，数据字典必须由专人（数据管理员）管理。数据管理员的职责就是维护和管理数据字典，保证数据字典内容的完整性和一致性。任何人，包括系统分析师、软件设计师、程序员，如果要修改数据字典的内容，都必须通过数据管理员。数据管理员还要负责把数据字典的最新版本及时通知有关人员。

11.5 面向对象分析方法

OOA 的基本任务是运用 OO 方法，对问题域进行分析和理解，正确认识其中的事物及它们之间的关系，找出描述问题域和系统功能所需的类和对象，定义它们的属性和职责，以及它们之间所形成的各种联系。最终产生一个符合用户需求，并能直接反映问题域和系统功能的 OOA 模型及其详细说明。

OOA 模型独立于具体实现，即不考虑与系统具体实现有关的因素，这也是 OOA 和面向对象的设计（Object-Oriented Design, OOD）的区别所在。OOA 的任务是“做什么”，OOD 的任务是“怎么做”。

11.5.1 用例模型

从用户的角度来看，他们并不想了解系统的内部结构和设计，他们所关心的是系统所能提供的服务，这就是用例方法的基本思想。用例方法是一种需求合成技术，从获取到的要求和期望中进行整理与提炼，从而建立用例模型。在 OOA 方法中，构建用例模型一般需要经历 4 个阶段，分别是识别参与者、合并需求获得用例、细化用例描述和调整用例模型，其中前三个阶段是必需的。

1. 用例图的元素

用例是一种描述系统需求的方法，使用用例的方法来描述系统需求的过程就是用例建模。在用例图中，主要包括参与者、用例和通信关联三种元素，如图 11-8 所示。

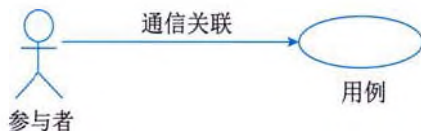


图 11-8 用例图中的基本元素

(1) 参与者。参与者是指存在于系统外部并与系统进行交互的任何事物，既可以是使用系统的用户，也可以是其他外部系统和设备等外部实体。

(2) 用例。用例是在系统中执行的一系列动作，这些动作将生成特定参与者可见的价值结果。也就是说，用例表示系统所提供的服务，它定义了系统是如何被参与者所使用的，它描述的是参与者为了使用系统所提供的某一完整功能而与系统之间发生的一段对话。

(3) 通信关联。通信关联表示的是参与者和用例之间的关系，或用例与用例之间的关系。箭头表示在这一关系中哪一方是对话的主动发起者，箭头所指方是对话的被动接受者，箭尾所

指方是对话的主动发起者。如果不想强调对话中的主动与被动关系，可以使用不带箭头的关联实线。在用例模型中，信息流不是由通信关联来表示的。该信息流是默认存在的，并且是双向的，它与箭头所指的方向没有关系。

2. 识别参与者

参与者是与系统交互的所有事物，该角色不仅可以由人承担，还可以是其他系统和硬件设备，甚至是系统时钟。

(1) 其他系统。当系统需要与其他系统交互时，其他系统就是一个参与者。例如，对某企业的在线教育平台系统而言，该企业的 OA 系统就是一个参与者。

(2) 硬件设备。如果系统需要与硬件设备交互，硬件设备就是一个参与者。例如，在开发集成电路（Integrated Circuit, IC）卡门禁系统时，IC 卡读写器就是一个参与者。

(3) 时钟。当系统需要定时触发时，时钟就是一个参与者。例如，开发在线测试系统中的“定时交卷”功能时，就需要引入时钟作为参与者。

3. 合并需求获得用例

将参与者都找到之后，接下来就是仔细地检查参与者，为每一个参与者确定用例。首先，要将获取到的需求分配给与其相关的参与者，以便可以针对每个参与者进行工作，而无遗漏；其次，进行合并操作。在合并之前，要明确为什么要合并，知道了合并的目的，才可能选择正确的合并操作。合并后，将产生用例。将识别到的参与者和合并生成的用例，通过用例图的形式整理出来，以获得用例模型的框架，如图 11-9 所示。

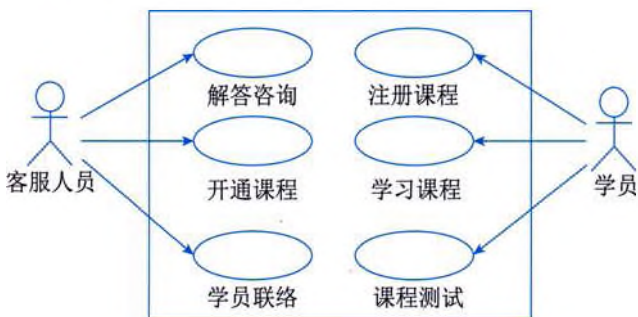


图 11-9 用例图示例

在确定用例的过程中，需要注意以下问题：

(1) 用例命名。用例的命名应该注意采用“动词（短语）+ 名词（短语）”的形式，例如，“开通课程”和“课程测试”等。而且，最好能够对用例进行编号，这也是实现需求跟踪管理的重要技巧，通过编号可以将用户的需求落实到特定的用例中去。

(2) 不能混淆用例和用例所包含的步骤。例如，“开通课程”功能要经过验证学员信息、检查学员权限、保存开通记录、修改课程选修人数等步骤才能完成，在系统中这些步骤不能作为单独的功能对外提供，它们只是一个用例所包含的事件流，或是用例的子功能。

(3) 注意区分业务用例和系统用例。当针对整个业务领域建模时，需要使用业务用例，其

中会涉及大量的人工活动，例如，在线教育平台系统中有一项重要工作是“编写教材”，这就是业务用例，是信息系统无法完成的。信息系统作为整个业务系统的一部分，只负责实现系统的部分功能，因此，只需要识别出系统用例，而不需考虑业务用例。

4. 细化用例描述

用例建模的主要工作是书写用例规约（Use Case Specification），而不是画图。用例模板为一个给定项目的所有人员定义了用例规约的结果，其内容至少包括用例名、参与者、目标、前置条件、事件流（基本事件流和扩展事件流）和后置条件等，其他的还可以包括非功能需求和用例优先级等。

用例描述通常包括以下几个部分：

（1）用例名称。用例名称应该与用例图相符，并写上其相应的编号。

（2）简要说明。对用例为参与者所传递的价值结果进行描述，应注意语言简要，使用用户能够阅读的自然语言。

（3）事件流。事件流是指当参与者和系统试图达到一个目标时所发生的一系列活动，也就是用例所完成的工作步骤。在编写时应注意使用简单的语法，主语明确，语义易于理解；明确写出“谁控制球”，也就是在事件流描述中，让读者直观地了解是参与者在控制还是系统在控制；从俯视的角度来编写，指出参与者的动作，以及系统的响应；描述用户意图和系统职责，而不叙述具体的行为和技术细节，特别是有关用户界面的细节。

执行一个用例的事件流有多种可能的路线，其中主事件流（基本事件流）是指能够满足目标的典型的成功路径，主事件流通常不包括任何条件和分支，符合大多数人的期望，从而更容易理解和扩展；备选事件流（扩展事件流）也称为备选路径，是完成用例可能出现失败的情况、分支路径或扩展路径，为了不影响用例活动清晰的主线，将这些分支处理全部抽取出来作为备选事件流。

另外，事件流的编写过程也是可以分阶段迭代进行的，对于优先级高的用例花更多的时间，更加细化；对优先级低的使用例可以先简略地将主事件流描述清楚，备选事件流留待以后处理。对于一些事件流较为复杂的，可以在用例描述中引用顺序图、状态图和通信图等手段进行描述。

（4）非功能需求。因为用例所涉及的非功能需求通常很难在事件流中进行表达，因此单列为一小节进行描述。在非功能需求的描述方面，一定要注意使其可度量和可验证。否则，就容易流于形式，形同摆设。

（5）前置条件和后置条件。前置条件是执行用例之前必须存在的系统状态，如果前置条件不满足，则用例无法启动；后置条件是用例执行完毕系统可能处于的一组状态。一旦用例成功执行，可能会导致系统内部某些状态的变化。而某些用例也可能没有前置条件或后置条件。如果在当前阶段不容易确定前置条件或后置条件，则可以在以后再细化。

（6）扩展点。如果包括扩展（或包含）用例，则写出扩展（或包含）用例名，并说明在什么情况下使用。

（7）优先级。说明用户对该用例的期望值，为以后的开发工作确定先后顺序。可以采用满意度/不满意度指标进行说明，例如，设置为1~5的数值。

表 11-2 是图 11-9 中“开通课程”的用例描述，这些内容不一定要一次完成。如果需要调整用例模型，则在调整用例模型之后，还可以修改和细化用例描述。

表 11-2 用例描述示例

-
1. 用例名称
开通课程（UC02）。
 2. 简要说明
为用户开通学习课程的权限，将其标记为“学员”，同时修改所选修课程的选修人数。
 3. 事件流
 - 3.1 主事件流
 - （1）客服人员向系统发出“开通课程”请求。
 - （2）系统要求客服人员选择开通课程的类型（软考、考研、专业课程、自学考试、Java 课程）。
 - （3）客服人员做出选择后，系统显示相应界面，让客服人员输入信息，并自动根据权限规则生成权限。
 - （4）客服人员输入学员的相关信息，包括学员用户名、所交费用、交费时间、选修课程名称。
 - （5）系统确认学员所交费用和所选修课程的规定费用一致。
 - （6）系统将所输入的信息存储建档，开通学员课程权限。
 - 3.2 备选事件流
 - （1）如果学员所交费用少于所选修课程的规定费用，则显示所选修课程的规定费用，并要求客服人员选择修改或取消输入。
 - 客服人员选择取消输入，则结束用例，不做存储建档工作。
 - 客服人员选择修改用户所交费用后，转到（5）。
 - （2）如果学员所交费用多于所选修课程的规定费用，则显示多余的费用数量，并要求客服人员选择是转换为学习币还是退还给学员。
 - 客服人员选择转换为学习币，则把多余的费用转换为学习币，记入学员账户中，转到（5）。
 - 客服人员选择退还给学员，转到（5）。
 - （3）如果学员所选修的课程超出了系统规定的选修人数，则提示客服人员，结束用例。
 4. 非功能需求
无特殊要求。
 5. 前置条件
客服人员登录在线教育平台系统。
 6. 后置条件
修改学员权限，修改课程选修人数。
 7. 扩展点
无。
 8. 优先级
最高（满意度 5，不满意度 5）。
-

5. 调整用例模型

在建立了初步的用例模型后，还可以利用用例之间的关系来调整用例模型。用例之间的关系主要有包含、扩展和泛化，利用这些关系，把一些公共的信息抽取出来，以便于复用，使得用例模型更易于维护。